INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA

PROJECT REPORT

COURSE TITLE: BACHELOR OF TECHNOLOGY

SUBJECT: PYTHON PROGRAMMING LABORATORY

PROJECT TITLE

INVENTORY AND RECORD TRACKER

NAME: 1. Soumyadeep Mitra (12025052003143)

2. Archisman Sinharoy (12025052003144)

3. Sumit Basak (12025052003145)

4. Anushka Chatterjee (12025052003146)

5. Chandrima Palit (12025052003149)

ROLL NO.: 131,132,133,134,135

SECTION: B

SUBMITTED TO: Python Laboratory

DEPARTMENT NAME: ELECTRONICS AND COMMUNICATION ENGINEERING

ACADEMIC SESSION: 2026–2027

DATE OF SUBMISSION: 29.10.26

Abstract

This project presents a Python-based Inventory and Record Tracker designed to maintain item details, monitor available stock, and record stock-in and stock-out activities in a structured manner. The application is intended for small laboratories, college departments, shops, and offices where inventory information is often maintained manually. It provides item registration, quantity updates, search and filtering, low-stock identification, transaction history, and CSV import/export. A graphical interface can be developed with Tkinter, while Python dictionaries and classes organise the records and the csv module stores data persistently.

The demonstration described in this report uses a small illustrative inventory dataset. Its sample counts and totals are examples for explaining the workflow and are not measurements from a real organisation. Validation rules prevent negative quantities, blank item identifiers, and invalid numeric entries. The tracker can reduce repetitive calculations and improve visibility of stock movement, but its reliability depends on accurate data entry, regular backups, and appropriate access control.

Keywords

Inventory management; stock tracking; record keeping; Python; Tkinter; CSV; data validation; low-stock alert; transaction history.

Table of Contents

1. Introduction
2. Problem Statement and Objectives
3. Software and System Requirements
4. System Design and Methodology
5. Implementation of Major Modules
6. Results and Discussion
7. Testing and Validation
8. Limitations and Future Scope
9. Conclusion
10. References
11. Appendix: CSV Format and Execution

1. Introduction

Inventory refers to the materials, products, equipment, or supplies held by an organisation. Keeping accurate records of item names, quantities, unit prices, and stock movement is important for avoiding shortages, overstocking, and misplaced records. When information is maintained only in notebooks or scattered spreadsheets, updating stock after each transaction can become slow and inconsistent.

The Inventory and Record Tracker is a Python desktop application concept that brings these tasks into one interface. Each item is assigned a unique item ID and is associated with a name, category, quantity, reorder level, and unit price. Stock-in and stock-out actions update the current quantity and create a dated transaction record. Search and summary views help users find items and identify stock that needs replenishment.

2. Problem Statement and Objectives

Problem statement: Develop a simple, structured inventory application that stores item records, validates updates, tracks stock movement, and produces useful summaries without relying on manual calculations.

The principal objectives are:

- Create, view, search, edit, and remove inventory item records.
- Record stock received and stock issued, while preventing negative stock.
- Display current quantity and identify items at or below their reorder level.
- Validate item IDs, text fields, prices, and quantities before saving changes.
- Import and export inventory data using CSV files.
- Maintain a transaction log for traceability of stock-in and stock-out operations.
- Summarise total item count, total units, and estimated inventory value.

3. Software and System Requirements

Component	Purpose
Python 3	Core program logic and data processing
Tkinter / ttk	Desktop graphical user interface
csv module	Reading and writing inventory and transaction CSV files
datetime	Recording the date and time of stock transactions
NumPy / Pandas (optional)	Extended summaries and tabular filtering
Matplotlib (optional)	Charts for category quantities or inventory value
Python IDLE or any Python IDE	Editing and running the application

The program can run on a desktop computer with Python 3 installed. Tkinter is commonly bundled with Python; optional third-party packages are required only if advanced analysis or charts are implemented.

4. System Design and Methodology

4.1 Input data and schema

Each inventory record contains item_id, item_name, category, quantity, reorder_level, and unit_price. The item ID uniquely identifies a product or material. Quantity and reorder level are non-negative whole numbers, while unit price must be numeric and non-negative. Transaction records contain transaction_id, item_id, transaction_type, quantity, date_time, and remarks.

4.2 Data preparation and validation

The application loads saved records from CSV files or starts with a small demonstration dataset. It checks required fields, verifies that item IDs are unique, and converts numeric values to the appropriate type. Invalid rows should be reported rather than silently accepted. Before a stock-out operation is committed, the requested quantity is compared with available stock.

4.3 Processing workflow

1. Load existing inventory records or create demonstration records.
2. Display the item list and allow the user to search or select an item.
3. Validate any new item, edit, stock-in, or stock-out request.
4. Update the item quantity and append a transaction log entry.
5. Recalculate inventory totals and low-stock indicators.
6. Save the current records to CSV and allow export when required.

4.4 Main calculations

Current quantity = opening quantity + total stock received − total stock issued.

Estimated item value = current quantity × unit price.

Estimated total inventory value = sum of estimated item values for all valid items.

Low-stock condition: current quantity ≤ reorder level. This condition is an alert for review; it does not automatically place a purchase order.

5. Implementation of Major Modules

Module / function	Functionality
InventoryItem class	Stores item fields and validates item data
add_item() / update_item()	Creates a new item or updates an existing record
stock_in() / stock_out()	Changes quantities and checks stock availability
validate_record()	Checks required fields and numeric limits
search_items() / filter_items()	Finds items by ID, name, or category
low_stock_items()	Returns items at or below the reorder level
load_csv() / save_csv()	Imports and exports inventory records
record_transaction()	Stores a history entry for each stock movement
refresh_dashboard()	Updates summary counts and table views

5.1 Interface layout

The main window can include an inventory table, search box, category filter, and buttons for Add Item, Edit, Delete, Stock In, Stock Out, Import CSV, and Export CSV. Summary indicators show the number of distinct items, total units, low-stock items, and estimated inventory value. A separate transaction-history view helps users review previous changes.

5.2 Validation rules

Item ID and item name cannot be blank. Item IDs must be unique. Quantity and reorder level cannot be negative. A stock-out quantity must be positive and cannot exceed available quantity. Unit price must be zero or greater. The application should ask for confirmation before deleting a record.

6. Results and Discussion

6.1 Demonstration inventory

The following illustrative data demonstrates how the tracker can summarise a small inventory. These values are examples for the report; actual results will depend on the CSV file and the transactions entered by the user.

Item ID	Item name	Category	Quantity	Reorder level	Unit price (₹)
IT001	LED	Electronics	40	10	15.00
IT002	Resistor pack	Components	8	10	25.00
IT003	Breadboard	Components	15	5	80.00
IT004	USB cable	Accessories	4	8	120.00
IT005	Notebook	Stationery	30	10	45.00

Indicator	Illustrative value
Distinct inventory items	5
Total units in stock	97
Items at or below reorder level	2
Estimated inventory value	₹ 3,415.00

The resistor pack and USB cable are at or below their reorder levels and should appear in the low-stock list. The estimated inventory value is calculated by multiplying each item's available quantity by its unit price and summing the results.

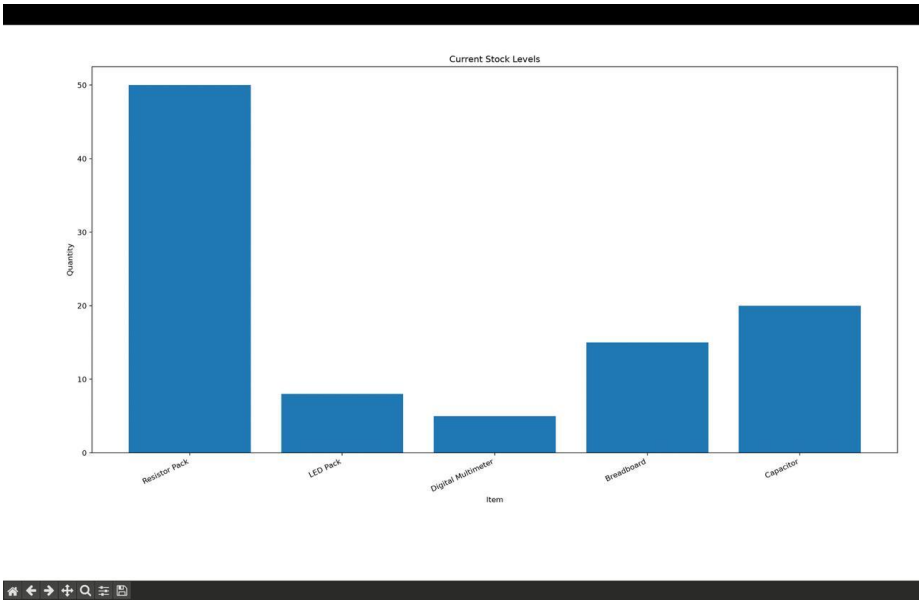
6.2 Example stock transaction

Transaction ID	Item ID	Type	Quantity	Resulting quantity	Remarks
TX001	IT002	Stock In	10	18	New delivery received
TX002	IT001	Stock Out	5	35	Issued for lab use

After a stock-in transaction, the available quantity increases; after a stock-out transaction, it decreases. Each operation should be stored in the transaction history so that a user can review when and why the quantity changed.

6.3 Discussion

The tracker consolidates item information and stock movement into a consistent workflow. Search and category filtering make records easier to locate, while reorder-level checks highlight possible shortages. CSV storage is simple and portable, although it is less suitable than a database when many users must edit records simultaneously. The displayed inventory value is an estimate based on the entered unit prices and should not be treated as an audited financial valuation.



7. Testing and Validation

Test case	Expected behaviour	Expected result
Add valid item	Accept unique ID and valid fields	Item appears in inventory
Duplicate item ID	Reject duplicate identifier	Validation message shown
Negative quantity	Reject quantity below zero	Record not saved
Stock out above available quantity	Prevent overselling/over-issuing	Quantity remains unchanged
Stock-in operation	Increase available quantity	Quantity and transaction log update
Low-stock check	Flag quantity ≤ reorder level	Item appears in alert list
CSV import with missing column	Report invalid file structure	Import cancelled or bad rows reported
Export inventory	Write current records to CSV	File can be reopened

8. Limitations and Future Scope

Limitations:

- CSV files do not provide strong multi-user concurrency or relational constraints.
- The accuracy of reports depends on correct and timely stock entry.
- The basic design may not include user accounts, permissions, or an audit-proof history.
- The inventory value is only as reliable as the stored unit prices and valuation method.
- The system does not automatically connect to suppliers, invoices, or barcode scanners.

Future scope:

- Use SQLite or another database for stronger data integrity and larger inventories.
- Add login, role-based permissions, backups, and audit logs.
- Integrate barcode or QR-code scanning for faster item identification.
- Create supplier, purchase order, and invoice modules.
- Generate PDF/Excel reports and date-wise stock movement summaries.
- Add configurable alerts for low stock, expiry dates, and slow-moving items.
- Support multiple storage locations and synchronised multi-user access.

9. Conclusion

The Inventory and Record Tracker project demonstrates how Python can support organised stock management and record keeping. The proposed workflow combines item records, input validation, stock-in and stock-out operations, low-stock alerts, transaction history, and CSV import/export. The illustrative examples show how current quantities and estimated inventory value can be calculated consistently. The application can be extended with a database, stronger access control, barcode scanning, and automated reporting. For dependable real-world use, records should be backed up regularly and all stock movements should be entered promptly.

10. References

- Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/
- Python Software Foundation. csv — CSV File Reading and Writing. https://docs.python.org/3/library/csv.html
- Python Software Foundation. tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html
- Pandas Development Team. pandas Documentation. https://pandas.pydata.org/docs/
- Matplotlib Development Team. Matplotlib Documentation. https://matplotlib.org/stable/

11. Appendix: CSV Format and Execution

Suggested inventory CSV headers (exact spelling): item_id, item_name, category, quantity, reorder_level, unit_price.

item_id	item_name	category	quantity	reorder_level	unit_price
IT001	LED	Electronics	40	10	15.00
IT002	Resistor pack	Components	8	10	25.00
IT003	Breadboard	Components	15	5	80.00

Suggested transaction CSV headers: transaction_id, item_id, transaction_type, quantity, date_time, remarks.

transaction_id	item_id	transaction_type	quantity	date_time	remarks
TX001	IT002	Stock In	10	2026-10-29 10:30	New delivery received
TX002	IT001	Stock Out	5	2026-10-29 11:15	Issued for lab use

Execution procedure

7. Install Python 3 and open the project source file in IDLE or another Python IDE.
8. If optional charts or data analysis are used, install the required packages.
9. Run the module and choose the sample inventory or import a CSV file.
10. Use the item and stock transaction controls to update the inventory.
11. Review low-stock items and transaction history, then export the updated CSV.

Example optional package installation command: py -m pip install pandas matplotlib

Note: The values in this report are illustrative. Replace them with the actual application output and screenshots after running and testing the final program.